Step-by-Step Guide — ADS-B Station with Telegram Alerts

This guide shows how to build, from scratch, a station that **receives aircraft signals** (ADS-B / Mode S on 1090 MHz), **feeds ADS-B Exchange**, and **sends Telegram alerts** whenever an aircraft from your own list is detected.

> **How to read this guide:** anything between `<` and `>` is a **placeholder** you must replace with your own value. Example: `<YOUR_TOKEN_HERE>`. Never post your real values in forums, chat groups or public repositories.

---

1. What you will need

Item	Notes
Raspberry Pi 3 (1 GB) or newer	A Pi 3 works, but memory is tight
microSD card, 16 GB or larger	Good quality (A1/A2 class)
Official 5V 2.5A Raspberry Pi power supply	A weak supply causes freezes and SD card corruption
RTL-SDR V3 (or a clone with an RTL2832U chip)	Software-defined radio receiver
1090 MHz band-pass filter	Reduces interference from other frequencies
1090 MHz antenna	The higher and clearer the view, the better the range
Cables and adapters (SMA)	Depending on your antenna and filter
Local network (Wi-Fi or Ethernet) with internet	Ethernet is more stable
Telegram account	To create the bot and receive alerts

---

2. How the system works

```
Antenna (1090 MHz)
   |
   v
1090 MHz filter
   |
   v
RTL-SDR (radio signal -> digital data)
   |
   v
Raspberry Pi
   |-- readsb ............ decodes and writes /run/readsb/aircraft.json
   |-- tar1090 ........... local web map (http://<PI_IP>/tar1090)
   |-- adsbexchange-feed . sends your data to the ADS-B Exchange network
   '-- radar-bot ......... reads aircraft.json and alerts you on Telegram
```

`readsb` is the heart of the system: it refreshes `aircraft.json` every second with every aircraft currently received. The other three services only consume that file or the `readsb` data port. Each service runs on its own under **systemd**, so everything comes back by itself after a power outage.

---

3. Hardware assembly

1. Connect the **antenna to the 1090 MHz filter**.
2. Connect the filter output to the **RTL-SDR** input.
3. Plug the RTL-SDR into a **USB port of the Raspberry Pi**. If possible, use a short USB extension cable to keep the dongle away from the Pi's heat and noise.
4. Place the antenna as **high as possible**, with a clear view of the sky, away from other electronics.

> **Tip:** antenna placement is the single biggest factor in range. No software setting makes up for a badly placed antenna.

---

4. Preparing the Raspberry Pi

4.1 Flash the operating system

1. On your computer, install **Raspberry Pi Imager** (raspberrypi.com/software).
2. Choose **Raspberry Pi OS Lite (64-bit or 32-bit)**. The Lite version has no desktop and saves memory.
3. Click the gear icon (advanced settings) and set:
   - Hostname, for example `adsb`
   - **Enable SSH** (password or key authentication)
   - Username: `<YOUR_USER>` and a strong password: `<YOUR_STRONG_PASSWORD>`
   - Wi-Fi details, if you will not use Ethernet

    - Your time zone
4. Write to the microSD card, put it in the Pi and power it on.

4.2 Find the IP address and connect over SSH

The Pi's IP appears in the device list of your router. Then connect:

```
ssh <YOUR_USER>@<PI_IP>
```

4.3 Update the system

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

> **Security:** always use a strong password and never expose the SSH port directly to the internet.

---

5. Install the decoder (readsb)

The *wiedehopf* project installer sets up `readsb` as a service and handles details such as disabling the DVB-T TV drivers that conflict with the RTL-SDR.

```
sudo bash -c "$(wget -nv -O - https://github.com/wiedehopf/adsb-scripts/raw/master/readsb-install.sh)"
```

If the installer asks you to reboot, do so. Then confirm:

```
sudo systemctl status readsb
```

You should see `Active: active (running)`.

5.1 Set the receiver location

The position is used to compute distances and is required for MLAT to work well. Edit the configuration file:

```
sudo nano /etc/default/readsb
```

Find the line that defines `RECEIVER_OPTIONS` and `DECODER_OPTIONS`, and add your position to the decoder options:

```
--lat <YOUR_LATITUDE> --lon <YOUR_LONGITUDE>
```

Use decimal degrees (format example: `-30.123456`). Save (`Ctrl+O`, `Enter`, `Ctrl+X`) and restart the service:

```
sudo systemctl restart readsb
```

> **Privacy:** if you do not want to expose the exact location of your home, use approximate coordinates. Keep in mind that MLAT depends on an accurate position, so being too vague can hurt it.

5.2 Check that data is arriving

```
cat /run/readsb/aircraft.json | head -c 500
```

You should see JSON with an `aircraft` key. The list may be empty if no aircraft are in range yet.

---

6. Install the local map (tar1090)

```
sudo bash -c "$(wget -nv -O - https://github.com/wiedehopf/tar1090/raw/master/install.sh)"
```

When it finishes, open this in a browser on the same network:

```
http://<PI_IP>/tar1090
```

You will see the aircraft your antenna receives in real time.

---

7. Feed ADS-B Exchange

This step sends your antenna's data to the global ADS-B Exchange network. It is optional for the bot, but it contributes to the community.

```
curl -L -o /tmp/axfeed.sh https://www.adsbexchange.com/feed.sh
sudo bash /tmp/axfeed.sh
```

The installer will ask for:

- **Latitude and longitude** of the antenna (decimal degrees)
- **Altitude** of the antenna in meters
- **Station name** (`<YOUR_STATION_NAME>`), with no spaces or special characters

Confirm that the service is up:

```
sudo systemctl status adsbexchange-feed
```

To check that your feed is being received and that MLAT is synchronized, visit:

```
https://www.adsbexchange.com/myip/
```

> **Normal:** right after boot you may see “Name resolution failed” messages in the feeder log. That is just DNS not being ready yet; it reconnects on its own within seconds.

---

8. Create the Telegram bot

8.1 Create the bot and get the token

1. In Telegram, search for **@BotFather** and start a chat.
2. Send `/newbot` and answer the questions (bot name and username).
3. BotFather replies with the **token**, something like `<YOUR_TOKEN_HERE>`. Keep it somewhere safe.

> **Warning:** the token gives full control of the bot. If it leaks, revoke it in **@BotFather -> /mybots -> your bot -> API Token -> Revoke current token**.

8.2 Create the group and get the Chat ID

1. Create a Telegram group and **add your bot** to it.
2. Send any message in the group.
3. On the Pi, query the bot’s updates (replacing the token):

```
curl "https://api.telegram.org/bot<YOUR_TOKEN_HERE>/getUpdates"
```

1. In the response, look for `"chat":{"id":` and note the number. For groups it is usually **negative**, for example `<YOUR_CHAT_ID>` in the format `-1234567890`.

---

9. Install the alert bot

9.1 Create the folder and install the dependency

```
mkdir -p ~/radar_bot
cd ~/radar_bot
sudo apt install -y python3-requests
```

9.2 Put the files in the folder

The required files are in **Appendix A** (bot code) and **Appendix B** (aircraft list and service). There are two ways to get them onto the Pi:

- **Option A (recommended):** copy them from your computer with `scp`, using the file package that accompanies this guide:

```
scp radar_bot.py aeronaves.json radar-bot.service <YOUR_USER>@<PI_IP>:~/radar_bot/
```

- **Option B:** create each file with `nano` and paste the content. Be careful: pasting large text into a terminal can mangle Python indentation. After pasting, always validate (step 9.4).

9.3 Create the credentials file (.env)

```
cd ~/radar_bot
nano .env
```

Content (replace with your own values):

```
TG_TOKEN=<YOUR_TOKEN_HERE>
TG_CHAT_ID=<YOUR_CHAT_ID>
TG_LANG=en
```

`TG_LANG` sets the language of the Telegram messages: `pt` (default) or `en`. The line is optional.

Protect the file so only your user can read it:

```
chmod 600 .env
```

9.4 Validate the files

```
python3 -m py_compile radar_bot.py && echo "Python OK"
python3 -m json.tool aeronaves.json > /dev/null && echo "JSON OK"
```

If either one fails, fix it before moving on.

9.5 Manual test

```
python3 radar_bot.py
```

You should receive “ADS-B Radar Online!” on Telegram. Stop the test with `Ctrl+C`.

---

10. Run the bot as a service (systemd)

The service makes the bot start at boot and restart if it crashes. If your user is not `pi`, adjust the `User=` line and the paths in the file before installing.

```
sudo cp ~/radar_bot/radar-bot.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable radar-bot
sudo systemctl start radar-bot
sudo systemctl status radar-bot
```

You should see `active (running)`. To follow the logs live:

```
sudo journalctl -u radar-bot -f
```

---

11. Adding and editing aircraft

The whole list lives in `~/radar_bot/aeronaves.json`. The bot reloads this file by itself every 5 minutes, **with no restart needed**.

```
nano ~/radar_bot/aeronaves.json
```

Each aircraft is one block:

```
{"nome": "DISPLAY NAME", "callsign": "PREFIX", "hex": ["HEX_CODE"]}
```

(The field names `nome`, `callsign` and `hex` are kept as they are in the code: `nome` is the display name.)

Important rules:

- `callsign` is the registration prefix **without the hyphen** (example: `PR-ABC` becomes `PRABC`).
- `hex` is always a **list**, even with a single code. If an aircraft has more than one hex, list them all: `["AAAAAA", "BBBBBB"]`.
- If you do not know the hex, use an empty list `[]` and detection will rely on the callsign only.
- Blocks are separated by a **comma**; the **last** block has no comma.
- Always use straight double quotes (`"`).
- Avoid repeating the same callsign in two entries: only the last one counts.

After editing, validate:

```
python3 -m json.tool ~/radar_bot/aeronaves.json > /dev/null && echo "Valid JSON!"
```

To apply immediately without waiting 5 minutes:

```
sudo systemctl restart radar-bot
```

Where to find the hex code: sites such as adsbexchange.com and flightaware.com let you search by registration; the hex also shows up on your local map while the aircraft is airborne.

---

12. Final test: full restart

Confirm that everything comes back on its own after a power outage:

```
sudo reboot
```

Wait about 90 seconds, reconnect and check:

```
sudo systemctl status readsb adsbexchange-feed radar-bot
```

All three should be `active (running)`, and you should receive “ADS-B Radar Online!” on Telegram without typing anything.

---

13. Everyday commands

What to do	Command
Check a service’s status	`sudo systemctl status radar-bot`
Restart the bot	`sudo systemctl restart radar-bot`
Stop / start the bot	`sudo systemctl stop radar-bot` / `sudo systemctl start radar-bot`
Follow the bot’s logs live	`sudo journalctl -u radar-bot -f`
Edit the aircraft list	`nano ~/radar_bot/aeronaves.json`
Validate the list	`python3 -m json.tool ~/radar_bot/aeronaves.json > /dev/null && echo OK`
Check memory usage	`free -h`
Check disk space	`df -h`
Find the Pi’s IP	`hostname -I`
Safely reboot the Pi	`sudo reboot`

---

14. Common problems

The bot sends nothing on Telegram Check `sudo journalctl -u radar-bot -n 30`. Common causes: wrong token or chat_id in `.env`, the bot was not added to the group, or the Pi has no internet.

The bot service fails to start (`status=1/FAILURE`) Almost always a wrong path in `radar-bot.service` or a missing `.env`. Check that the files are in the folder named in `WorkingDirectory` and `ExecStart`.

“Expecting property name enclosed in double quotes” when validating the JSON A missing comma or `}` in some block, or a trailing comma after the last item. The error often points to the line **after** the real problem; look at the previous line too.

The map shows “Problem fetching data from the server: JSON.parse unexpected end of data” The map read `aircraft.json` in the middle of a write. If it is occasional, refresh the page. If it persists, check that `readsb` is running (`sudo systemctl status readsb`), that the web server is up (`sudo systemctl status lighttpd`), and memory (`free -h`). A reboot usually fixes it, but if the problem returns the Pi may be at its resource limit.

Few aircraft or short range Review antenna position, cable, filter and receiver gain. In urban areas, trying a fixed gain instead of automatic sometimes improves reception.

“Name resolution failed” messages in the feeder right after boot Normal; DNS was not ready yet and the feeder reconnects on its own.

---

15. Security summary

- Never put the token, password or chat_id inside the code or in shared screenshots.
- Keep `.env` at permission `600` and out of any public repository.
- Change the Pi user’s default password and use a strong one.
- Do not expose SSH directly to the internet; if you need remote access, use a VPN.
- If any secret leaks, revoke it and generate a new one immediately.
- Keep the system updated: `sudo apt update && sudo apt full-upgrade`.

Appendix A — Bot code (`radar_bot.py`)

Save this file as `~/radar_bot/radar_bot.py`. It **contains no credentials**: the token and chat_id are read from the `.env` file. Set `TG_LANG=en` in `.env` for English alerts.

```
import requests
import time
import json
import socket
import os
import logging
from pathlib import Path

# --- CONFIGURAÇÕES GERAIS ---
BASE_DIR = Path(__file__).resolve().parent
AIRCRAFT_JSON = "/run/readsb/aircraft.json"
AERONAVES_JSON = BASE_DIR / "aeronaves.json"
ENV_FILE = BASE_DIR / ".env"

INTERVALO_LOOP = 10          # segundos entre leituras do aircraft.json
INTERVALO_RECARGA_DB = 300   # segundos entre recarregar aeronaves.json (5 min)
JANELA_ANTISPAM = 1800       # 30 minutos sem repetir alerta da mesma aeronave

# --- LOGGING ---
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger("radar_bot")


# --- CARREGAR VARIÁVEIS DE AMBIENTE (.env simples, sem depender de lib externa) ---
def carregar_env(caminho: Path) -> None:
    if not caminho.exists():
        return
    with open(caminho, "r") as f:
        for linha in f:
            linha = linha.strip()
            if not linha or linha.startswith("#") or "=" not in linha:
                continue
            chave, valor = linha.split("=", 1)
            os.environ.setdefault(chave.strip(), valor.strip())


carregar_env(ENV_FILE)

TOKEN = os.environ.get("TG_TOKEN")
CHAT_ID = os.environ.get("TG_CHAT_ID")

# Idioma das mensagens do Telegram: "pt" (padrão) ou "en". / Telegram message language.
IDIOMA = os.environ.get("TG_LANG", "pt").strip().lower()
if IDIOMA not in ("pt", "en"):
    IDIOMA = "pt"

TEXTOS = {
    "pt": {
        "online": "🚀 *Radar ADS-B Online!*\n📍 *IP:* `{ip}`\nMonitorando {n} identificadores
cadastrados.",
        "alvo": "📡 *Alvo Identificado!*",
        "ident": "✈️ *Identificação:*",
        "callsign": "🆔 *Callsign:*",
        "hex": "📟 *Hex (ICAO):*",
        "alt": "📏 *Altitude:*",
        "vel": "🚀 *Velocidade:*",
        "rumo": "🧭 *Rumo:*",
        "squawk": "📻 *Squawk:*",
        "mapa": "Abrir Mapa Local",
    },
    "en": {
        "online": "🚀 *ADS-B Radar Online!*\n📍 *IP:* `{ip}`\nMonitoring {n} registered
identifiers.",
        "alvo": "📡 *Target Identified!*",
        "ident": "✈️ *Identification:*",
```

```
        "callsign": "🆔 *Callsign:*",
        "hex": "📟 *Hex (ICAO):*",
        "alt": "📏 *Altitude:*",
        "vel": "🚀 *Speed:*",
        "rumo": "🧭 *Track:*",
        "squawk": "📻 *Squawk:*",
        "mapa": "Open Local Map",
    },
}
T = TEXTOS[IDIOMA]

if not TOKEN or not CHAT_ID:
    log.error(
        "TG_TOKEN e/ou TG_CHAT_ID não configurados (TG_TOKEN and/or TG_CHAT_ID not set). "
        "Copie .env.example para .env / Copy .env.example to .env."
    )
    raise SystemExit(1)


# --- BANCO DE AERONAVES (externo, recarregável sem reiniciar o bot) ---
def carregar_banco_aeronaves(caminho: Path) -> dict:
    """
    Lê aeronaves.json (lista de {nome, callsign, hex}) e monta um dicionário
    único de busca por callsign OU hex, ambos em maiúsculas.
    O campo "hex" aceita tanto uma string única quanto uma lista de códigos
    (útil quando a mesma aeronave tem mais de um hex registrado/histórico).
    """
    banco = {}
    try:
        with open(caminho, "r", encoding="utf-8") as f:
            lista = json.load(f)
        for item in lista:
            nome = item.get("nome", "").strip()
            callsign = item.get("callsign", "").strip().upper()
            hex_valor = item.get("hex", [])

            if isinstance(hex_valor, str):
                hex_lista = [hex_valor] if hex_valor.strip() else []
            elif isinstance(hex_valor, list):
                hex_lista = hex_valor
            else:
                hex_lista = []

            if not nome:
                continue
            if callsign:
                banco[callsign] = nome
            for h in hex_lista:
                h = h.strip().upper()
                if h:
                    banco[h] = nome
        log.info(f"Banco de aeronaves carregado: {len(lista)} aeronaves cadastradas.")
    except FileNotFoundError:
        log.error(f"Arquivo {caminho} não encontrado.")
    except json.JSONDecodeError as e:
        log.error(f"Erro ao ler {caminho} (JSON inválido): {e}")
    return banco


def enviar_telegram(mensagem: str, tentativas: int = 3) -> bool:
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    payload = {"chat_id": CHAT_ID, "text": mensagem, "parse_mode": "Markdown"}
    for tentativa in range(1, tentativas + 1):
        try:
            r = requests.post(url, data=payload, timeout=10)
            if r.status_code == 200:
                return True
            log.warning(f"Telegram respondeu {r.status_code}: {r.text[:200]}")
        except requests.exceptions.RequestException as e:
```

```
            log.warning(f"Tentativa {tentativa}/{tentativas} falhou: {e}")
        if tentativa < tentativas:
            time.sleep(2)
    log.error("Falha ao enviar mensagem ao Telegram após todas as tentativas.")
    return False


def get_ip() -> str:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
        s.close()
        return ip
    except OSError:
        return "127.0.0.1"


def montar_mensagem(nome_exibir: str, callsign: str, hex_id: str, ac: dict, meu_ip: str) -> str:
    alt = ac.get("alt_baro", "N/A")
    velocidade = ac.get("gs", "N/A")       # ground speed, em nós
    rumo = ac.get("track", "N/A")          # direção em graus
    squawk = ac.get("squawk", "N/A")

    return (
        f"{T['alvo']}\n"
        f"{T['ident']} `{nome_exibir}`\n"
        f"{T['callsign']} `{callsign if callsign else 'N/A'}`\n"
        f"{T['hex']} `{hex_id}`\n"
        f"{T['alt']} {alt} ft\n"
        f"{T['vel']} {velocidade} kt\n"
        f"{T['rumo']} {rumo}°\n"
        f"{T['squawk']} `{squawk}`\n\n"
        f"[{T['mapa']}](http://{meu_ip}/tar1090)"
    )


def limpar_avisados(avisados: dict, agora: float) -> dict:
    return {k: v for k, v in avisados.items() if agora - v < JANELA_ANTISPAM}


def main():
    avisados = {}
    meu_ip = get_ip()
    banco_aeronaves = carregar_banco_aeronaves(AERONAVES_JSON)
    ultima_recarga_db = time.time()
    ultima_limpeza = time.time()

    log.info(f"Bot iniciado (Filtro Exclusivo) no IP: {meu_ip}")
    time.sleep(2)

    enviar_telegram(T["online"].format(ip=meu_ip, n=len(banco_aeronaves)))

    while True:
        agora = time.time()

        # Recarrega o banco de aeronaves periodicamente, sem reiniciar o bot
        if agora - ultima_recarga_db > INTERVALO_RECARGA_DB:
            banco_aeronaves = carregar_banco_aeronaves(AERONAVES_JSON)
            ultima_recarga_db = agora

        # Limpa entradas antigas do dicionário de antispam
        if agora - ultima_limpeza > 3600:
            avisados = limpar_avisados(avisados, agora)
            ultima_limpeza = agora
            log.info(f"Limpeza de 'avisados' concluída. {len(avisados)} entradas ativas.")

        try:
            if not os.path.exists(AIRCRAFT_JSON):
```

```
                log.warning(f"Aguardando {AIRCRAFT_JSON}...")
                time.sleep(INTERVALO_LOOP)
                continue

            with open(AIRCRAFT_JSON, "r") as f:
                dados = json.load(f)

            for ac in dados.get("aircraft", []):
                callsign = ac.get("flight", "").strip().upper()
                hex_id = ac.get("hex", "").upper()

                if not hex_id:
                    continue

                nome_exibir = banco_aeronaves.get(callsign) or banco_aeronaves.get(hex_id)
                if not nome_exibir:
                    continue

                if hex_id not in avisados or (agora - avisados[hex_id] > JANELA_ANTISPAM):
                    msg = montar_mensagem(nome_exibir, callsign, hex_id, ac, meu_ip)
                    if enviar_telegram(msg):
                        avisados[hex_id] = agora

        except json.JSONDecodeError:
            # O readsb pode estar no meio de uma escrita do arquivo; ignora e tenta de novo no
próximo ciclo
            log.debug("aircraft.json incompleto/corrompido nesta leitura, ignorando ciclo.")
        except Exception as e:
            log.error(f"Erro inesperado no loop: {e}")

        time.sleep(INTERVALO_LOOP)


if __name__ == "__main__":
    main()
```

Appendix B — Supporting files

B.1 `aeronaves.json` (example with fictitious data)

The names, prefixes and codes below are **made up**, only to show the format. Replace them with the aircraft you want to monitor.

```
[
  {"nome": "EXAMPLE HELICOPTER 01", "callsign": "PPXXX", "hex": ["AAAAAA"]},
  {"nome": "EXAMPLE RESCUE 02", "callsign": "PRYYY", "hex": ["BBBBBB", "BBBBBC"]},
  {"nome": "EXAMPLE NO HEX", "callsign": "PSZZZ", "hex": []}
]
```

B.2 `.env.example`

```
# Copy to ".env" and fill in. NEVER publish the ".env" file.
TG_TOKEN=<YOUR_TOKEN_HERE>
TG_CHAT_ID=<YOUR_CHAT_ID>
# Telegram message language: pt (default) or en
TG_LANG=en
```

B.3 `radar-bot.service`

Adjust `User` and the paths if your user is not `pi`.

```
[Unit]
Description=ADS-B Radar Bot (Telegram)
After=network-online.target readsb.service adsbexchange-feed.service
Wants=network-online.target

[Service]
Type=simple
User=<YOUR_USER>
WorkingDirectory=/home/<YOUR_USER>/radar_bot
ExecStart=/usr/bin/python3 /home/<YOUR_USER>/radar_bot/radar_bot.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```